ONTARIO TECH UNIVERSITY
FACULTY OF SCIENCE, COMPUTER SCIENCE

VCLab April 22, 2026

Assessing Generalization Catastrophic Forgetting in Multi-domain Continual Learning

by

Alan Zheng Faisal Qureshi Kourosh Davoudi

Abstract: *As neural networks grow, training a single general purpose model becomes a more financially viable and simpler alternative to training multiple specialized models for individual tasks. However, a key challenge remains: how can we learn across multiple domains of tasks while preventing catastrophic forgetting? Most continual learning methods constrain tasks to within a specific domain, but their effectiveness across multiple distinct domains remains unclear. In this work, we evaluate several approaches including, Orthogonal Gradient Descent (OGD), Selective Forgetting-Aware Optimization (SFAO), LoRA, and ShareLoRA across multiple domains. Our results show that while SFAO improves on OGD in certain settings, OGD demonstrates a stronger ability to prevent forgetting especially across multiple domains in different contexts. Additionally, although LoRA and ShareLoRA are not explicitly designed in preventing catastrophic forgetting, they present a strong ability in preventing catastrophic forgetting while also generalizing learning across seen and unseen tasks. These findings highlights a critical importance in how we define continual learning tasks to span across a single domain or across varying domains, and demonstrate methods that may perform differently depending on this distinction.*

1 Introduction

Recent progress in zero-shot learning has demonstrated significant accuracy across multiple contexts. However, the challenge of continual Learning, adapting a model to learn new tasks without degrading knowledge of previous tasks (catastrophic forgetting), remains an active area of research. Continual zero-shot Learning is the process of continually learning new latent spaces in a dataset while specifically being able to evaluate on different classes; specifically, for image classification, training on a specific set of classes while accurately evaluating on a different set of classes which may or may not be in similar context.

Much of the focus of continual learning has been about learning of new classes across a single domain (dataset) which is arguably more important when looking at further improving the abilities of specialized models doing well with similar inputs. However, in real-world applications: obtaining data under consistant conditions is often impossible; For images: when data lighting conditions change, method of obtaining image (drone, satellite, body cameras) and for text: different context in writing (comments, blogs, storytelling). A key significant benefit of focusing on multiple datasets is the development of general-purpose vision language model or even extending further than the capacity of this research is Artificial General Intelligence.

1.1 Multi-Domain Task Incremental Learning (MTIL) & Task-Free Learning (TFL)

MTIL evaluates unseen and seen classes across different domains instead of the standard **Class Incremental Learning**, which only evaluates unseen and seen separated classes within a single specific domain. Our paper addresses continual training in Multiple Domains without providing specific tasks ids for which class label belongs to, which is classified as Task-Free Learning by [10] and [11]

1.2 CLIP (Vision Language Model)

The model used to train and evaluate our continual learning methods is the CLIP model developed by [8]; Regular vision models aren't able to extend their class labels without having to changing the architecture or head, and risk damaging the underlying representations of the model.

Rather than just training an image model with label, training a language model in tandem with a vision model allows for the large corpus of data to be compared with our vision model and so be able to evaluate on various classes, which is important if we wish to expand our class size especially for our task of continual learning.

1.3 Continual Learning Methods

1.3.1 Distillation

The concept of distillation is the process of using a ”Teacher Model” to ”distill” it's knowledge onto the ”Student Model”, or in literal terms, the student learns from the logits of the teacher model. Usually the teacher model is of a larger size in terms of complexity and raw parameter count such that the smaller student model is able to learn a more general representation of the knowledge the teacher has.

Distillation loss is described as such:

$$CE(y, \bar{y}) = -\sigma\left(\frac{y}{\tau}\right) \cdot \log \sigma\left(\frac{\bar{y}}{\tau}\right)$$

where y are the student logits, $\bar{y}$, are the teacher logits, σ, is the softmax function, and τ, is the temperature.

To preserve the relationships learned by the teacher model are transferred to the student model, τ is introduced to divide across all logits to scale the larger logits while keeping the same relationships.

1.3.2 Weight Ensembling

In order to smooth out training and retain the original model's weight distribution we employ weight ensembling:

$$\hat{\theta}_t = \begin{cases} \theta_0 & \text{if } t = 0 \\ \frac{1}{t+1}\theta_t + \frac{t}{t+1}\hat{\theta}_{t-1} & \text{every } I \text{ iterations} \end{cases}$$

Instead of using a modified learning rate between the two weights, blending of the weights are done uniformly to prioritize preserving knowledge from earlier trained knowledge [1]. This equation also scales the newly computed parameters, θ_t by a factor of $\frac{1}{t+1}$ while the old parameters, $\hat{\theta}_{t-1}$ by a factor of $\frac{t}{t+1}$, essentially giving

more weight to the previous parameter and less to the new parameters as training goes on. While this prevents learning on long training sessions, it also prevents large damaging shifts (catastrophic forgetting); And with long training, the model usually should not need to learn large changes, but rather tweak smaller changes.

1.3.3 LoRA (Low-Rank Adaption)

LoRA replaces standard model weights with a trainable set of decomposition matrices.

Let $W \in \mathbb{R}^{m \times n}$ be the original weight matrix of the model, let $A \in \mathbb{R}^{r \times n}$ and $B \in \mathbb{R}^{m \times r}$ be the decomposition matrices such that $BA \in \mathbb{R}^{m \times n}$.

$$h = Wx + BAx$$

When merging with original weights:

$$h = (W + BA)x$$

To ensure stability when accounting for differing ranks, the merged weights are scaled by a factor of $\frac{\alpha}{r}$, where α is a hyperparameter chosen to either increase or decrease training speed. The factor is scaled by r as well to ensure that when increasing the ”complexity” of learning, learning rate is stabilized to prevent large unwanted changes.

2 Related Works

2.1 Data Replay

Many works have proposed data replay as a method to mitigate catastrophic forgetting by ”retraining” old tasks and replaying old memory. Ding *et al.* [7] have presented that depending on the stored data, data replay can: improve old tasks' performance even compared to when the model finished training on such task, mitigate the catastrophic forgetting problem by reducing the loss of performance on newer tasks, and even with data replay catastrophic forgetting can still occur. One finding from [7] mentions how much data replay mitigates forgetting depends

on two factors, ordering of learning new tasks, and the noise/signal learned from later tasks. Ding *et al.* mentions how higher correlation between the two tasks enhances learning and mitigation, however on our task of learning multiple tasks across diverse domains which may not align does not guarantee this condition, and such data replay may not guarantee improvements and may potentially enlarge the forgetting by mixing noises.

2.2 Parameter Expansion

Another method of mitigating catastrophic forgetting is with the use of expansion of the base model with either simply more parameter counts or in this case adding multiple backbones, classification head, etc. to enable the model to evaluate across multiple different tasks. *Wu et al.* [9] have presented a novel approach to involve numerous dynamic backbones using a Self-Controled Dynamic Expansion Model (SCDEM). One of the benefits specifically for this model is the use of the Collaborative Optimization Mechanism (COM) which optimizes the various signals across all backbones (preventing loss in information when relying on a single backbone), and using experts to determine whether the current task requires creation of new experts for evaluating the decision boundary. This prevents the need for specific task id's present when switching between different experts and allows for use of backbones and experts across all previous histories. While this method is novel towards continual learning and parameter expansion techniques; The downside of Parameter Explosion, where eventually accumulating tasks will eventually expand the model too much to where it is potentially impossible to make use of or if limited in parameter expansion, will again encounter the catastrophic forgetting problem.

Mainly due to this fact of a theoretical limit to parameter expansion techniques, we have decided to focus on other methods to mitigate the catastrophic forgetting problem.

2.3 Class Incremental Learning

Singh *et al.* uses Selective Forgetting-Aware Optimization to prevent forgetting by gating updates from the cosine similarity of the current task and the previous tasks [6]. While Singh *et al.* prevents degradation of previous gradients, Singh *et al.* only takes into account of tasks within a single dataset and does not learn the different representations across different datasets in different domains. However

Selective Forgetting-Aware (SFAO), greatly improves forward transfer of similar tasks and preventing forgetting of the current task; either accepting changes if the current gradient computed aligns with the current task, projecting parts of current gradient onto old gradient if new gradient aligns (cosine similarity) above a certain threshold, or completely disregard all information if the new gradient drops lower than a certain threshold (usually $\tau < 0$). This does not align with our task of attempting to learn multiple representations of different datasets which may not all align in one direction and may align in the opposite.

3 Methods

3.1 Implementation of Distillation

Rather than using a more complex larger model for our teacher model, we used the same architecture as our original pretrained CLIP model throughout the training process as the teacher model. Alternatively, we do not use the previous updated model, $\hat{f}_{t-1}$, for our teacher model, instead we keep the original model, $\hat{f}_0$, to act like the teacher to distill its information; this results in a better prevention of shifts in the distribution towards a specific task.

To further prevent distribution shifts towards a specific domain, the reference dataset chosen when distilling its logits is of a more general representation of the multiple task.

3.2 Orthogonal Gradient Descent (OGD)

When training weights of a model, we keep track of all the partial derivatives($\frac{\partial L}{\partial \theta}$)/gradients of all the weights within a layer with respective to the Loss L:

$$\nabla L_{(x,y)}(w) = \left[\frac{\partial L}{\partial \theta_0}, ..., \frac{\partial L}{\partial \theta_n}\right]$$

where $\frac{\partial L}{\partial \theta_i}$ is the gradients for a single layer within our model, which includes the weight matrix and bias vectors, and n is the number of layers we track within our model.

At every k iteration we save the previous list of gradients: $\nabla L_{(x,y)}(w)_k$.

After every dataset, we decompose each list of gradients into SVD basis matrices:

$$\nabla L_{(x,y)}(w)_k = U\Sigma V$$

We then use the singular value matrix (which is a diagonal matrix), Σ, to reduce the noisy directions from our list of saved gradients. We calculate total enery by: total energy $= \sum_{i=0}^{n} \sigma_i^2$, where i, is the index of the of the diagonal element and n is the total columns. We sum up the energy to some percentage of the total by taking $\sum_{i=0}^{k} \sigma_i^2 = \alpha \cdot$ total energy, where $k < n$; This gives us the highest k columns with the highest singular values, or the highest importance columns. Not only does keeping the largest singular values with the highest energy reduce take out unneeded information, this makes the saving of all the gradient basis' significantly smaller specifically for such a large model like CLIP.

3.3 ShareLoRA

To further extend the capabilities of LoRA, we have implemented ShareLoRA [12] which further reduces required amount of parameter update counts of at least 0.81% while at the same time providing even more generalization abilities.

ShareLoRA is implemented across all layers with the same dimensions, where with standard LoRA the whole model consists of B_1A_1, B_2A_2, ..., B_nA_n matrix products, where n is the number of layers the model has. Instead we use B_1A_1, B_1A_1, ..., B_1A_1 for all layers with shared dimensions, this significantly reduces redundant updates that are potentially harmful and contribute to catastrophic forgetting especially for similar domain shifts when learning new tasks.

ShareLoRA is further extended for the transformer layers in this model; where query, key, value, and output projection layers are present. Each of these layers receives their own low rank matrices (q: A_q, B_q, k: A_k, B_k, v: A_v, B_v, o: A_o, B_o).

3.4 Metrics

3.4.1 Backward Transfer

The backwards transfer metric best represents how much the model has lost accuracy(information) on a specific task i. Calculating only on trained datasets, this metric directly measures the catastrophic forgetting the model experiences for a specific dataset.

$$BWT = \frac{1}{T-1}\sum_{i=1}^{T-1}(a_{T,i} - a_{i,i})$$

where T, is the total number of datasets the model has learned, $a_{T,i}$ is the accuracy of the model at the end on task i, and after learning all tasks: T, $a_{i,i}$, represents the accuracy of task i, when completed training on task i.

3.4.2 Forward Transfer

The forward transfer metric measures how well the model has learned (+ or -) on a specific task/domain before it has been explicitly trained on that task. This directly measure the zero-shot transfer ability the model has across multiple domains. It is defined as:

$$FWT = \frac{1}{T-1}\sum_{i=1}^{T-1}(a_{i,i} - a_{i-1,i})$$

where $a_{i,i}$, is the accuracy evaluation on task i, when completed training on task i, and $a_{i-1,i}$, is the accuracy of task i, when completed on the task beforehand $t-1$.

4 Experimental Setup

We employ CLIP (ViT-B/16) as the model to train; training both vision and text encoders, with each dataset/task being trained for 5000 iterations each. We have chosen 5000 iterations specifically as a balance between training time, dataset size, and a large enough learning time for the model; training each task by epoch may end up favoring larger datasets than smaller ones, and potentially misrepresent knowledge loss, especially when paired with distillation or weight ensembling every n iterations. In addition, Smaller iterations ended up with less of an consistent trend as the model learns new tasks. The optimizer used is AdamW (which gives us good generalization and preventing overfitting with weight decay) and the learning rate scheduler employed is cosine decay to continuously vary our learning rate.

4.1 Lora

For the hyperparameter for LoRA: all training is done with a low rank $r = 8$, and alpha, α to regularize the update of the layers with we use 16, chosen due to smaller ranks displaying a large reduction in performance.

4.2 Zero-Shot Continual Learning (ZSCL)

ZSCL consists of many methods combined to combat catastrophic forgetting while also enabling zero-shot abilities (generalizability) across unseen datasets [1]. This combination consists of: Weight Ensembling and Distillation and for further explanation in the next section.

4.3 Orthogonal Gradient Descent

To prevent too large of a gradient accumulation size on disk we chose 10 gradients to be saved distributed throughout the 5000 iterations; every 500 iterations we save the current gradient computation, and at the end decompose the stack of gradients into their singular values, and keep the top 0.97% of the basis' based on their energy. This method provides a good balance in the amount of information we save for each task and the resulting gradient file size saved.

4.3.1 Reference Model

In order to preserve the zero-shot capabilities of the original CLIP model, we need a general dataset to anchor our model to a relative "general" subspace. Therefore, to prevent distribution shifts away from the general subspace, no matter how minor, we do not use the previous learned task model and use the initial frozen CLIP model as the teacher, as that would prevent any deviation resulting in forgetting from the previous trained models.

4.3.2 Reference Dataset

Instead of distilling knowledge from the current task (which would induce bias from the task and not prevent forgetting), we use a reference dataset for both vision and language encoders. Specifically we use ImageNet and Conceptual Captions for the Vision and Text Transformers respectively; ImageNet is one of the largest datasets, 14 million images, for vision tasks and has a diverse set of

classes that is represented in the dataset. Conceptual Captions is a image - text paired dataset used to train vision language models like CLIP for this research, and contains 3 million images. These two datasets are a good basic and not highly specific dataset to ground the model as it learns new tasks.

5 Results and Discussion

Our main focus of these experiments are to prevent the degradation of information (accuracies) of all domains (tasks/datasets) particularly on those that have been previous trained on. Our secondary focus is the positive transfer of knowledge across different domains, particularky on those we have yet to be trained on.

5.1 Average Accuracy

Task	Base	DTD	DTD → MNIST	DTD → MNIST → EuroSAT	DTD → MNIST → EuroSAT → Flowers
Evaluated Datasets / Methods	All	All	All	All	All
ZSCL	65.86	68.60	**70.52**	70.29	75.70
ZSCL + OGD	65.86	**68.99**	69.91	67.51	74.72
ZSCL + LoRA	65.86	68.07	70.28	**72.82**	**76.61**
ZSCL + ShareLoRA	65.86	67.59	70.00	72.44	75.46
ZSCL + SFAO	65.86	65.88	53.92	49.14	70.17

Table 1: The average accuracy evaluated on all datasets: Aircraft, Caltech101, CIFAR100, DTD, EuroSAT, Flowers, Food, MNIST, OxfordPet, StandfordCars, ImageNet. Each task is trained sequentially from the previous task. The best results are highlighted in **bold**

Looking purely at the average accuracy of the model on all tasks, as shown in Table 1, the best-performing model across to use across these datasets is ZSCL with LoRA layer update presents the best overall performance on any of those datasets; that is if the specific sequence of training is done (DTD, MNIST, EuroSAT, Flowers) and that you don't continue training on different datasets.

Expectedly both OGD and SFAO performs with the lowest scores out of the rest of the methods; This is likely due to the fact that Table 1. evaluates on all datasets re-

gardless of whether they had learned them or not. For both OGD and SFAO, they are at this point ineffective at this point without the gradient information learned from the domains, and are potentially harmful and adding to the catastrophic forgetting.

While the average accuracy of the evaluation on all of our datasets tells us the best-performing model at the current position in time, t, it does not show the trends (particularly in datasets we have already learned) in forgetting and transfer learning which are important for continual learning and zero-shot learning.

5.2 Backward Transfer Scores

Task	DTD	DTD → MNIST	DTD → MNIST → EuroSAT	DTD → MNIST → EuroSAT → Flowers	-
Methods \ Evaluated Datasets	DTD	MNIST	EuroSAT	Flowers	Average
ZSCL	-2.18	-0.17	**-0.17**	0.00	-0.62
ZSCL + OGD	**-1.81**	**-0.04**	-0.28	0.00	**-0.53**
ZSCL + LoRA	-2.23	-0.11	-0.26	0.00	-0.65
ZSCL + ShareLoRA	-4.15	-0.40	-0.50	0.00	-1.26
ZSCL + SFAO	-4.89	-0.05	-0.35	0.00	-1.32

Table 2: The backward transfer scores only of the trained datasets (difference of final task vs current task). The task listed is the order the dataset has been trained on (Starting with the earliest task). Each task is trained sequentially from previous. The best results are highlighted in **bold**

As shown in table 2, Orthogonal Gradient Descent performs the best across most of the datasets and the best overall in the backward transfer scores, somewhat unexpectedly SFAO performs the worst out of the rest. This may be due to the fact of either the accept, λ_{accept} being too low, causing accepting changes that forgets information of the past by fully updating a gradient slightly towards the wrong direction. However with $\lambda_{accept} = 0.9$ makes change unlikely to forget information due to the cosine similarity between the two gradients needing to be very similar (0.9) to slightly change direction.

The primary reason SFAO underperforms compared to the rest may stem from how its gradients that oppose the previous directions are handled. SFAO assumes

the initial tasks represents a general subspace for all following tasks. However, for our setup each domain does not share contexts and therefore may not always be aligned on the same direction, even displaying conflict with each other. While OGD is less rigid in the condition that all tasks must never go against the grain, by still accepting orthogonal components. SFAO employs a much more rigid mechanism compared to OGD by explicitly measuring cosine similarity: (1) accept gradients if fully aligned, (2) project partially aligned gradients, (3) reject gradients that are conflict with previous gradients. Resulting in SFAO aggressively preventing learning and prevention of forgetting.

5.3 Forward Transfer Scores

Task	Base → DTD	DTD → MNIST	DTD → MNIST → EuroSAT	-
Methods \ Evaluated Datasets	MNIST	EuroSAT	Flowers	Average
ZSCL	5.78	0.22	-11.90	**-1.48**
ZSCL + OGD	**7.56**	**2.43**	-18.64	-2.16
ZSCL + LoRA	1.03	-3.28	**-8.80**	-2.76
ZSCL + ShareLoRA	0.70	-4.36	-11.29	-3.74
ZSCL + SFAO	9.03	-17.52	-49.19	-14.42

Table 3: The forward transfer scores only of the trained datasets. The task listed is the order the dataset has been trained on (Starting with the earliest task). Each task is trained sequentially from previous. The best results are highlighted in **bold**

Base ZSCL presents the best evidence for zero-shot capabilities (forward transfer) with the best average of forward transfer scores in Table 3. Adding both OGD and SFAO lowers this score, with SFAO drastically preventing zero-shot capabilities. Theoretically, this means that these techniques of preventing changes along any direction are harmful for ZSCL techniques: using a reference dataset and distillation which may require certain ”opposing” gradients to align the model to a general subspace rather than a tuned one.

For SFAO, with its main purpose to either align updates along the previous gradients or project onto orthogonal directions may finetune too much towards a specific task; Looking at Table 4. of ZSCL + SFAO accuracy of 98.83% on the EuroSAT dataset right after training, shows the best performance of that specific

dataset, however once we move into the next dataset (Flowers), we can see the forward transfer score for that dataset is also significantly less than all other methods. This describes that while SFAO is perfectly aligned to prevent forgetting and even improve performance of a specific subspace (EuroSAT), the result is the catastrophic forgetting of other domains.

ZSCL + ShareLora or LoRA performing worse on forward transfer scores in Table 3. compared to regular ZSCL is as expected, due to the lack of diverse representation LoRA layers can learn and update

5.4 Best Task Accuracy

Task	DTD	DTD → MNIST	DTD → MNIST → EuroSAT	DTD → MNIST → EuroSAT → Flowers	DTD → MNIST → EuroSAT → Flowers
Methods \ Evaluated Datasets	DTD	MNIST	EuroSAT	Flowers	Average
ZSCL	80.00	99.21	97.76	97.58	93.64
ZSCL + OGD	79.47	99.30	98.57	97.56	93.73
ZSCL + LoRA	79.10	99.37	98.50	**97.95**	93.73
ZSCL + ShareLoRA	77.98	99.29	98.33	97.38	93.25
ZSCL + SFAO	**81.06**	**99.56**	**98.83**	97.58	**94.26**

Table 4: The accuracy **only** on the current task right after completely training on current task: DTD, MNIST, EuroSAT, Flowers. Each task is trained sequentially from the previous task. The best results are highlighted in **bold**

SFAO displays the best performance on the current task right after learning that task based on Table 4. This unique case and application of SFAO makes sense when we look at the difference between OGD and SFAO, particularly in SFAO allowing the full gradient distribution shifts if the current tasks closely aligned with the previous. For example if the DTD dataset has similar distributions to the dataset that CLIP was pretrained on, then the new updates are completely accepted.

Unexpectedly base ZSCL performs worse than SFAO and even OGD (which is highly restrictive on distribution shifts); Based on the unique way both OGD and SFAO handle unaligned gradients, potentially noises within the training data severely affect the training much more than expected, and restricting the model's distribution shift is needed for a model like CLIP which already has learned a good foundation of image classification.

5.5 Total Number of Parameters

Methods	Update-able Weights	Total Weights	Percentage of Update-able Weights
ZSCL	151 M	151 M	100%
ZSCL + OGD	151 M	151 M	100%
ZSCL + LoRA	3.9 M	151 M	2.56%
ZSCL + ShareLoRA	1.2 M	151 M	0.81%
ZSCL + SFAO	151 M	151 M	100%

Table 5: The number of parameters required to update the model during training

Looking at compute efficiency across all methods, according to Table 5. LoRA and ShareLoRA significantly reduces the VRAM and RAM needed to compute the gradients during updates. However from our experiements, there was no significant improvement to training time nor saving model size due the nature of the implementation of LoRA.

6 Conclusions

As continual learning methods develop it is important to understand the nuances in how we define and evaluate continual learning methods. Specifically, we are interested in learning cross-domain datasets and evaluating the model across all datasets including trained and untrained datasets without providing the model any task IDs for the datasets using a model which has been pre-trained on a general task. This work evaluates multiple methods and specific implementations of those methods to evaluate the catastrophic forgetting each of these methods are susceptible to catastrophic forgetting.

The main problem of researching the reduction of catastrophic forgetting of the methods has resulted in Orthogonal Gradient Descent presenting the best evidence of preventing performance loss on tasks it has previously learned and which preserve information from previous stored gradients of the past gradients; Although additionally does not prevent loss of information not present in previous gradient updates, in this case of the dataset, used to pre-trained CLIP.

The findings challenge the evaluation metrics and specific continual learning tasks of these methods, especially for SFAO which does not incorporate different domains from varying datasets, but focuses on preventing loss on a single domain when learning new classes.

6.1 Future Work

6.1.1 Tasks

This study focused on only learning 4 learned tasks due to compute and time limitations; potentially cutting off important trends we may see when learning even more tasks. Future work may expand and result in further findings unique to longer training sessions.

Another factor that greatly affects catastrophic forgetting is the order in which the tasks were trained at, which were lightly touched upon in this research but did not fully test nor experimented to evaluate the findings of such affects.

6.1.2 Limitations of OGD

While OGD presents the best ability to prevent catastrophic forgetting, the accumulation of gradients may present another issue of its own, while a very small probability and depending on the size of the model (number of possible dimensions), the accumulating gradients if large enough can span all directions equally leaving the model to either forcefully apply updates against the grain of the previous gradients or none at all.

References

[1] Zangwei Zheng, Mingyuan Ma, Kai Wang, Ziheng Qin, Xiangyu Yue and Yang You. *Preventing Zero-Shot Transfer Degradation in Continual Learning of Vision-Language Models*. arXiv preprint arXiv:2303.06628, 2023

[2] Mehrdad Farajtabar, Navid Azizan, Alex Mott and Ang Li *Orthogonal Gradient Descent for Continual Learning* arXiv preprint arXiv:1910.07104, 2019

[3] Seokmin Ko *LoRA Is Slower Than You Think* arXiv preprint arXiv:2507.08833, 2025

[4] Wenxuan Zhang, Paul Janson, Kai Yi, Ivan Skorokhodov and Mohamed Elhoseiny *Continual Zero-Shot Learning through Semantically Guided Generative Random Walks* arXiv preprint arXiv:2308.12366, 2023

[5] Sriram Mandalika, Harsha Vardhan and Athira Nambiar *Replay to Remember (R2R): An Efficient Uncertainty-driven Unsupervised Continual Learning Framework Using Generative Replay* arXiv preprint arXiv:2505.04787, 2025

[6] Anika Singh, Aayush Dhaulakhandi, Varun Chopade, Likhith Malipati, David Martinez, and Kevin Zhu *Mitigating Forgetting in Continual Learning with Selective Gradient Projection* arXiv preprint arXiv:2603.26671, 2026

[7] Meng Ding, Jinhui Xu, and Kaiyi Ji *Provable Effects of Data Replay in Continual Learning: A Feature Learning Perspective* arXiv preprint arXiv:2602.02767, 2026

[8] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever *Learning Transferable Visual Models From Natural Language Supervision* arXiv preprint arXiv:2103.00020 , 2021

[9] Runqing Wu, Kaihui Huang, Hanyi Zhang, and Fei Ye *Self-Controlled Dynamic Expansion Model for Continual Learning* arXiv preprint arXiv:2504.10561 , 2025

[10] Rahaf Aljundi, Klaas Kelchtermans, and Tinne Tuytelaars *Task-Free Continual Learning* arXiv preprint arXiv:1812.03596, 2018

[11] Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu *A Comprehensive Survey of Continual Learning: Theory, Method and Application* arXiv preprint arXiv:2302.00487, 2023

[12] Yurun Song, Junchen Zhao, Ian G. Harris, Sangeetha, and Abdu Jyothi *ShareLoRA: Parameter Efficient and Robust Large Language Model Fine-tuning via Shared Low-Rank Adaptation* arXiv preprint arXiv:2406.10785, 2025

A An appendix

A.1 Final Accuracies on Trained Datasets

Task	DTD → MNIST → EuroSAT → Flowers	DTD → MNIST → EuroSAT → Flowers	DTD → MNIST → EuroSAT → Flowers	DTD → MNIST → EuroSAT → Flowers	DTD → MNIST → EuroSAT → Flowers
Methods \ Evaluated Datasets	DTD	MNIST	EuroSAT	Flowers	Average
ZSCL	**77.82**	97.65	<u>97.58</u>	99.04	93.02
ZSCL + OGD	<u>77.66</u>	<u>98.30</u>	97.56	<u>99.26</u>	**93.19**
ZSCL + LoRA	76.86	98.24	**97.95**	<u>99.26</u>	<u>93.08</u>
ZSCL + ShareLoRA	73.83	97.83	97.38	98.89	91.98
ZSCL + SFAO	76.17	**98.48**	<u>97.58</u>	**99.51**	92.93

Table 6: The accuracy on the listed task **after completely training all** these following tasks in order: DTD, MNIST, EuroSAT, Flowers. The best results are highlighted in **bold**, the next best results are <u>underlined</u>

Task	Base	DTD	DTD → MNIST	DTD → MNIST → EuroSAT	DTD → MNIST → EuroSAT → Flowers
Methods \ Evaluated Datasets	Trained Datasets Only	Trained Datasets Only	Trained Datasets Only	Trained Datasets Only	Trained Datasets Only
ZSCL	57.61	66.46	74.89	83.00	93.02
ZSCL + OGD	57.61	67.87	74.81	81.44	93.19
ZSCL + LoRA	57.61	64.89	73.48	84.59	93.08
ZSCL + ShareLoRA	57.62	64.55	72.84	83.27	91.98
ZSCL + SFAO	57.61	66.33	62.91	73.41	92.93

Table 7: Average accuracies across all trained datasets at each time the model has learned new task